Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Bu kayıt "Yapısal Programlama Uygulama-3"tür. Uygulamayı büyük olasılıkla bir araştırma görevlisi yaptı. Ahmet Hoca derse yetkiyi verip ayrıldı.
- Kaydın sesi çok düşük ve anlaşılmaz olduğu için bu özet büyük ölçüde ekran paylaşımındaki koddan çıkarıldı.
- Problem: bir şirketin çalışan bilgilerini saklayan bir veritabanı. Menü üzerinden çalışan ekleme, isme göre arama, kayıt silme, tüm kayıtları listeleme ve sonunda bütün dinamik belleği serbest bırakma işlemleri yapılacak. Aynı işlev iki farklı veri yapısıyla yazılıp karşılaştırılıyor: dinamik dizi ve bağlı liste (linked list).
- Struct'lar
  - `typedef struct job { char *Name; } JOB;` çalışanın meslek adı.
  - `typedef struct person { char *Name; int Age; JOB *job; } PERSON;` çalışanın adı, yaşı ve mesleği.
  - Struct'lar `typedef` ile kısaltılır, böylece her yerde `struct person` yazmak gerekmez.
- 1. Dizi tabanlı yaklaşım
  - `typedef struct database { PERSON **Data; int peopleCount; int databaseCount; } DATABASE;`
    - `Data`: `PERSON*` işaretçilerinden oluşan dinamik dizi.
    - `peopleCount`: kayıtlı kişi sayısı. `databaseCount`: dizinin kapasitesi.
  - `main` başlangıcı: `peopleCount = 0`, `databaseCount = 1`, `Data = (PERSON**) malloc(sizeof(PERSON*) * databaseCount)`.
  - Menü: `while (exitControl)` içinde `switch (choice)`. 1 ekleme, 2 listeleme, 3 arama, 4 silme, 0 çıkış (`exitControl = 0`), `default` yanlış seçim.
  - `PERSON *initNewPerson(char *Name, int Age, char *jobName)`:
    - `PERSON` ve `JOB` için `malloc`.
    - Ad ve meslek adı için `malloc((strlen(Name) + 1) * sizeof(char))`, sonra `strcpy`. `strlen` sondaki `'\0'` karakterini saymadığı için +1 eklenir.
    - Bu yüzden struct'taki `Name` yalnızca işaretçidir; string için ayrıca yer ayrılması gerekir.
    - `newPerson->job = newJob;` ve `return newPerson;`. İşaretçiyle üyelere `->` operatörüyle erişilir.
  - `void AddPerson(DATABASE *personDatabase, PERSON *newPerson)`:
    - `Data[peopleCount++] = newPerson;`
    - Dizi dolunca (`databaseCount == peopleCount`) kapasite iki katına çıkarılır: `databaseCount *= 2`, `Data = (PERSON**) realloc(Data, databaseCount * sizeof(PERSON*))`.
    - `realloc`, yer varsa alanı yerinde genişletir; yoksa başka bir yere kopyalayıp yeni adresi döndürür.
    - Veritabanı fonksiyona adresiyle (`&personDatabase`) gönderilir, çünkü içindeki alanlar değişecek.
  - `void PrintPerson(PERSON *person)`: `"Name: %s, Age: %d, JobName: %s"`, `person->job->Name`.
  - `void ListDatabase(DATABASE *db)`: bütün `Data[i]` için `PrintPerson`.

  - `int SearchDatabase(DATABASE *db, char *Name)`: `strcmp(Data[i]->Name, Name) == 0` ise “Person Found!” yazıp indisi döndürür. Bulamazsa `-1` döndürür (geçerli bir indis negatif olamaz).
  - `void FreePerson(PERSON *person)`: sırasıyla `person->Name`, `person->job->Name`, `person->job`, `person` serbest bırakılır.
  - `void DeleteDatabase(DATABASE *db, char *Name)`:
    * Önce `SearchDatabase` ile konum bulunur. `-1` ise “Person Not Found!”.
    * `FreePerson(Data[pos])`.
    * Silinen yerde boşluk kalmasın diye sonraki elemanlar bir sola kaydırılır, `peopleCount--`.
    * Alternatif: sıralama önemli değilse son elemanı silinen yere koymak. Böylece her şeyi kaydırmak gerekmez.
  - `void FreeDatabase(DATABASE *db)`: `peopleCount > 0` olduğu sürece son kişiyi `FreePerson` ile serbest bırakır, sonra `free(Data)`. Program `0` seçilince bunu çağırıp çıkar.

- 2. Bağlı liste (linked list) yaklaşımı

  - Yeni yapı: `typedef struct node { PERSON *Person; struct node *Next; } NODE;`. `Next`, bir sonraki düğümü gösterir; sonuncunun `Next`’i `NULL`’dır. `next` ile bütün liste dolaşılır.
  - Veritabanı artık yalnızca listenin başını gösteren `NODE *head` işaretçisidir (başlangıçta `NULL`).
  - `void AddPersonFirst(NODE **head, PERSON *newPerson)`: yeni düğümün `Next`’i eski `*head` olur, `*head` yeni düğüm olur. Baş işaretçisi değiştiği için `NODE **` (işaretçinin adresi) gönderilir.
  - Başa ekleme yapıldığı için listeleme ters sırada çıkar: 1, 2, 3 eklenince liste 3, 2, 1 yazdırıldı.
  - `void AddPersonLast(NODE **head, PERSON *newPerson)`:
    * Liste boşsa (`*head == NULL`) yeni düğüm baş olur.
    * Değilse `current->Next != NULL` olana kadar ilerlenir, son düğümün `Next`’ine yeni düğüm bağlanır. Sona eklenecek olduğu için `current` `NULL` olana kadar değil, `current->Next` `NULL` olana kadar gidilir.
  - `void ListDatabase(NODE *head)`: `current = head`, `while (current != NULL) { PrintPerson(current->Person); current = current->Next; }`. Baş değişmeyeceği için `head`’in kendisi gönderilir.
  - Arama: aynı şekilde liste dolaşılır, `strcmp` ile isim karşılaştırılır.
  - `void DeleteDatabase(NODE **head, char *Name)`:
    * `previous = NULL`, `current = *head`. İsim bulunana kadar `previous = current; current = current->Next;`.
    * `current == NULL` ise “Person Not Found!” ve `return`.
    * Silinen ilk düğümse (`previous == NULL`) `*head = current->Next`. Değilse `previous->Next = current->Next` (düğüm zincirden çıkarılır).
    * `FreePerson(current->Person)` ve düğümün kendisi serbest bırakılır.
  - Listenin tamamını serbest bırakırken bir sonraki düğümün adresi önce saklanır, sonra mevcut düğüm silinir. Aksi hâlde zincir kopar.

- Dizi ve bağlı liste karşılaştırması

  - Bağlı liste bellek el verdiği sürece büyür. Dizide dolunca iki katına çıkarıp yeniden yer ayırmak (`realloc`) gerekir.
  - Bağlı listede silmek için yalnızca ilgili düğüm zincirden çıkarılır, kaydırma gerekmez. Sıralı bir dizide silinen yer boş kalmasın diye bütün elemanlar kaydırılır.
  - Bağlı listede her eleman için ek bir `Next` işaretçisi tutulur, bu da ek bellek demektir.
  - Dizide indisi bilinen elemana doğrudan erişilir. Bağlı listede indis yoktur; elemanın yeri bilinse bile baştan `next, next, …` diye gidilir.
  - Çok ekleme ve silme olan durumlarda bağlı liste daha uygundur. Eleman sayısı sınırlı ve erişim sık ise dinamik dizi tercih edilebilir.

- Başlık dosyaları ve makrolar (ek konu)

  - `#include <stdio.h>`: derleyicinin kendi dizininde aranır. `#include "menu.h"`: önce programın bulunduğu dizinde aranır.

- Include koruması: `#ifndef MENU_H`, `#define MENU_H`, …, `#endif`. Aynı başlık birden fazla kez dahil edilirse tanımlar tekrar etmesin diye kullanılır.
- Makro: `#define MENU printf(...); printf(...);`. Satır sonundaki `\` makronun bir sonraki satırda devam ettiğini gösterir. Derlemeden önce (önişlemci aşamasında) `MENU` yazan yere bu ifade kopyalanır.
- `calculation.h` içinde `#define topla(x, y) ((x) + (y))` gibi makrolar. `header.c` dosyası `menu.h` ve `calculation.h`'yi dahil ederek bir hesap makinesi menüsü kurar (`switch` ile toplama, çıkarma, çarpma). Böylece programlar birbirinden bağımsız dosyalara ayrılır.

Hocanın Özellikle Vurguladığı Kısımlar

- Struct içindeki string işaretçiler için `strlen + 1` kadar ayrı yer ayırın ve `strcpy` ile kopyalayın.
- Serbest bırakma sırası: önce iç alanlar (isimler, iş), sonra struct'ın kendisi.
- Bağlı listede baş işaretçisi değişebilecekse fonksiyona işaretçinin adresini (`NODE **`) gönderin.
- Bağlı listeden silerken bir önceki düğümü (`previous`) takip edin.
- Sık ekleme-silme varsa bağlı liste, sık indisli erişim varsa dizi.

Kısa Tekrar Notları

- `PERSON **Data` + `peopleCount` + `databaseCount`, dolunca `databaseCount *= 2` ve `realloc`.
- String kopyalama: `p->Name = malloc(strlen(s) + 1); strcpy(p->Name, s);`
- Düğüm: `struct node { PERSON *Person; struct node *Next; }`.
- Başa ekleme: `n->Next = *head; *head = n;`. Sona ekleme: `while (cur->Next) cur = cur->Next; cur->Next = n;`.
- Silme: `previous->Next = current->Next` ya da baş için `*head = current->Next`.
- `#ifndef X / #define X / #endif` include koruması.

Detaylı Açıklamalar

Ders sonrası notlar

- **Kaydın anlaşılırlığı:** Ses çok düşük ve dağınık olduğu için altyazının büyük kısmı anlamsızdır. Kod ve akış ekran görüntülerinden doğrulandı. Altyazıdaki "çarpı çarpı çarpı…" diye kırpılmış tekrar (yaklaşık 00:26:17) `initNewPerson` içinde `malloc(sizeof(PERSON))` satırının yazıldığı ana denk gelir; ekranda kod okunabildiği için içerik kaybı yoktur.
- **`main`'deki giriş dizileri:** Ekranda ilk sürümde `int Name[100], jobName[100];` yazıyordu ve derleyici "incompatible pointer type" uyarısı verdi. gcc 15.2 bu durumu varsayılan ayarda derleme hatası sayar, `-std=gnu89` ile uyarı verir (denendi). İsimler için `char Name[100], jobName[100];` olmalıdır; bağlı liste sürümünde böyle düzeltildi.
- **`realloc` kullanımı:** `AddPerson`'da sonuç doğrudan `Data`'ya atanıyor ve `NULL` kontrolü yapılmıyor. Ders 6'da anlatıldığı gibi önce geçici bir işaretçiye alıp kontrol etmek daha güvenlidir.
- **`SearchDatabase`'in dönüş tipi:** İlk yazıldığında `void` tanımlanmış ama `return i;` içeriyordu; sonra `int` olarak düzeltildi. İndis döndüren fonksiyonun tipi `int` olmalıdır.
- **Dizide silmede iki hata:** Ekrandaki ilk sürümde (1) kişi bulunamadığında "Person Not Found!" yazıldıktan sonra fonksiyondan çıkılmıyor, `Data[-1]` serbest bırakılmaya çalışılıyordu; `return` eklenmelidir. (2) Kaydırma döngüsü `for (i = pos; i < peopleCount; i++) Data[i] = Data[i + 1];` biçimindeydi; son adımda `Data[peopleCount]` okunur, bu kayıtlı bir eleman değildir. Sınır `i < peopleCount - 1` olmalıdır. Düzeltilmiş hâli (derste yazılmadı) gcc ve AddressSanitizer ile test edildi: "Ali, Ayse, Veli" eklenip "Ayse" silinince "Ali, Veli" kaldı, olmayan isim için "Bulunamadi" yazıldı, bellek hatası çıkmadı.
  - `int pos = Search(db, n); if (pos == -1) { printf("Bulunamadi\n"); return; }`
  - `free(db->Data[pos]->Name); free(db->Data[pos]);` (derste ayrıca `job` alanı da serbest bırakılıyor)

  - `for (i = pos; i < db->peopleCount - 1; i++) db->Data[i] = db->Data[i + 1]; db->peopleCount--;`

- **Son elemanı silinen yere taşıma:** Bu yöntem kaydırma yapmadığı için $O(1)$'dir ama elemanların sırası değişir. Kayıtların sırası önemliyse kullanılmamalıdır.
- **Bağlı listede düğümün kendisi:** Silmede `FreePerson(current->Person)` yalnızca kişi verisini serbest bırakır. Düğüm de `malloc` ile oluşturulduğu için ayrıca `free(current)` çağrılmalıdır (ekranda bu satır yazılmaya başlanmıştı).
- **Makrolarda parantez:** `#define topla(x, y) x + y` gibi parantezsiz bir makro `2 * topla(3, 4)` ifadesinde `2 * 3 + 4 = 10` verir. Doğrusu `((x) + (y))` biçiminde parantezli yazmaktır; o zaman sonuç 14 olur.
- **Uygulamayı yapan kişi:** Kayıtta adı net anlaşılamadığı için özette adı verilmedi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.